

SkyGuard AI — Full Hackathon Execution Plan

Project: Intelligent Real-Time Anomaly Detection for AWS (Temperature, Pressure, Humidity)

Hackathon sprint: 24–48 hours | Audience: Beginners

1. Market Research

1.1 Existing Solutions

Solution	What it does	Limitation
IMD AWS QC System (India)	Range checks, step checks, temporal/spatial consis	Rule-based; evolving; no guaranteed error-free dat
WMO / Nordic QC Standards	Standardized gross error, step, internal consisten	Static thresholds; struggles with complex multivar
Met Éireann LSTM Autoencoder	Learns normal patterns; 99.6% accuracy on valid da	Research-stage; single-variable focus; not widely
Commercial AWS (Vaisala, Campbell Scientific)	Hardware QC, calibration alerts	Vendor-locked; expensive; weak cross-station intel
China CMA XGBoost QC	Multi-source rainfall anomaly detection	Heavy rainfall focus; needs radar/satellite; not T
LSTM-AE Flatline Detection (2024 research)	Detects stuck/frozen sensors	Narrow use case; not integrated with operational I

Bottom line: Most production systems still rely on threshold + rule-based QC. ML approaches exist in papers but are not operational at national scale in India, especially with explainability and real-time deployment.

1.2 Gaps & Unmet Needs

- Real-time intelligent QC — IMD monitors 24/7 but relies on manual maintenance cycles (3-day SLA for fixes); anomalies like Delhi's 52.9°C faulty reading (May 2024) reached public platforms before correction.
- Multivariate consistency — Temperature, pressure, and humidity are physically coupled; most QC treats them independently.
- Explainability gap — Forecasters need to know why a flag was raised (sensor fault vs. heatwave).
- Flatline / frozen sensor detection — Rule-based methods miss subtle stuck values.
- Predictive maintenance — No widespread "sensor health score" before total failure.
- Edge deployment — Rural AWS sites have power/GSM issues; cloud-only ML is impractical.
- Scalability across 700+ IMD AWS stations — Climatology missing for many stations (IMD's own QC docs note this).

1.3 Target Audience & Pain Points

Audience	Pain Point	SkyGuard Value
IMD / State Met Centres	Bad data leads to bad forecasts; manual QC overloa	Automated flags + confidence scores
Farmers / Agri-tech (127 agro-AWS in India)	Wrong temp/humidity leads to crop loss decisions	Trustworthy corrected streams
Aviation / Disaster Mgmt	Extreme readings trigger false alerts	Distinguish real events vs. sensor faults
Climate Researchers	Archival data polluted by undetected anomalies	Clean datasets with QC metadata
AWS Maintenance Teams	Reactive, site visits after failure	Predictive sensor health dashboard

India-specific context:

- ~707 AWS + 1,351 ARG stations in IMD network (as of training docs)

- Data received at Pune central servers via INSAT/GPRS
- Preventive maintenance is manual; sensor replacement only after drift detected
- 2024 Delhi incident proved public trust risk from unverified AWS readings

1.4 Trends, Technologies & Stats

Trend Relevance
LSTM Autoencoders for QC State-of-art for temporal anomaly detection (Met É
Explainable AI (SHAP/LIME) 10% of your hackathon score — judges want interpre
Edge AI (ESP32) 5% energy efficiency score; deploy QC at station I
Robust statistics (MAD, double-standardization) 2025 research for correlated multivariate meteorol
Digital Twin / Self-healing networks Grand Challenge alignment — auto-correct + alert m
Global weather data market ~\$2.5B+ and growing; India investing under Ministr

Key stat for your pitch: IMD's own QC improved accuracy to ~95% after rule-based checks — meaning ~5% of data can still be problematic. AI can target that long tail.

2. Execution Plan (24–48 Hour Sprint)

Phase 0: Pre-Hackathon Prep (Do BEFORE the clock starts)

- Download IMD open data or use NOAA/NCEI AWS datasets
- Pre-build a synthetic anomaly injector (spikes, flatlines, drift, comm gaps)
- Set up GitHub repo, Docker, and a one-page architecture diagram
- Assign roles (see Section 2.3)
- Prepare 3 demo scenarios (including the 55°C example from the problem statement)

2.1 48-Hour Roadmap

Hours 0–4: Ideation

- Problem framing & scope lock (2h)
- Dataset finalization (2h)

Hours 4–8: Design

- Architecture & API design (2h)
- UI wireframes (2h)

Hours 8–21: ML Core

- Data preprocessing pipeline (4h)
- Baseline rules + Isolation Forest (3h)
- LSTM-AE or multivariate model (6h)
- SHAP explainability layer (3h)

Hours 20–28: Backend & Frontend

- FastAPI/Node API + WebSocket (4h)

- Dashboard (Plotly/Chart.js) (4h)

Hours 28–35: Integration

- End-to-end pipeline (4h)
- Demo scenarios + injected faults (3h)

Hours 36–48: Polish

- Pitch deck + README (4h)
- Testing on eval dataset (4h)
- Rehearsal + backup video (4h)

2.2 24-Hour Compressed Roadmap

Hours	Milestone	Deliverable
0–2	Ideation	Scope doc, 3 anomaly types prioritized
2–6	Data + Baseline	Clean pipeline, rule-based QC, synthetic anomalies
6–12	ML Core	Isolation Forest + simple LSTM-AE OR multivariate
12–16	API + Dashboard	Live stream simulation, alert cards
16–20	Explainability	SHAP feature importance + root-cause labels
20–22	Demo prep	3 scripted scenarios, metrics table
22–24	Pitch + buffer	Slides, 3-min demo video backup

2.3 Team Roles (4–5 members ideal)

Role	Owner	Responsibilities
Team Lead / Pitcher	1 person	Story, slides, demo script, judge Q&A
ML Engineer	1 person	Model training, anomaly injection, metrics
Backend Engineer	1 person	API, WebSocket stream, database
Frontend Engineer	1 person	Dashboard, maps, alert UI
Data + QA	1 person	Dataset, preprocessing, test cases, README

Solo/duo hackathon? Lead = ML + pitch; Partner = full-stack + dashboard.

3. Minimum Viable Product (MVP)

MVP Definition (Must-Have for Demo)

INPUT: T (°C) + P (hPa) + RH (%) time series (CSV upload OR simulated live stream)

CORE:

1. Rule-based pre-filter (range/step)
2. ML anomaly detector (Isolation Forest/LSTM)
3. Multivariate consistency check
4. Root-cause classifier (4–5 categories)

OUTPUT:

- Real-time alert feed
- Severity (Low/Med/High/Critical)
- Confidence score (0–100%)
- SHAP-based explanation (top 2 features)
- Sensor health score per station
- Dashboard with time-series + flags

MVP Feature Checklist (mapped to scoring)

Feature Hackathon Weight MVP Priority
Anomaly detection accuracy 20% P0
Innovation (multivariate + explainability) 25% P0
Real-time stream simulation 15% P0
SHAP/LIME explanations 10% P0
Multi-station scalability (architecture) 10% P1
Deployability (Docker/README) 10% P1
Dashboard UI 5% P1
Edge AI demo (ESP32 mock) 5% P2 (stretch)
Data imputation Optional P2 (stretch)

What to CUT if time runs out

- Deep learning → use Isolation Forest + rules
- ESP32 edge → show architecture slide only
- Imputation → show formula on slide, skip code
- Multi-station map → 2 stations is enough

4. Unique Selling Proposition (USP)

Core USP Statement

"SkyGuard AI is a self-aware weather data guardian that tells you not just THAT something is wrong, but WHY — distinguishing a Delhi heatwave from a broken thermistor in real time."

Differentiators

USP Why it wins
1 Physics-Informed Multivariate QC Uses known T–P–RH relationships (e.g., dew point c
2 Explainable Root-Cause Taxonomy Labels: SPIKE, FLATLINE, DRIFT, COMM_ERROR, PHYSIC
3 Neighbor-Aware Spatial Check Compares station vs. 3 nearest AWS (simulated) — d
4 Sensor Health Score (0–100) Predictive maintenance dashboard — aligns with Gra
5 Hybrid Edge-Cloud Architecture Rules on ESP32 (low power) + ML in cloud — hits En
6 Confidence-Calibrated Alerts Every alert shows % confidence + SHAP top features

One-liner for judges

"Traditional QC asks: 'Is this value in range?' SkyGuard asks: 'Does this value make physical and spatial sense right now?'"

5. Three Execution Perspectives (Tech Stacks)

Option 1: Beginner-Friendly Full Stack (Recommended for first hackathon)

Best for: Teams comfortable with web dev, limited ML depth

Frontend: HTML + CSS + JavaScript + Chart.js/Plotly.js

Backend: Node.js + Express + Socket.io (real-time)

Database: MongoDB (alerts, station metadata)

ML: Python microservice (Flask/FastAPI)

- Pandas, NumPy, Scikit-learn
- Isolation Forest + Z-score rules
- SHAP for explainability

Data: CSV upload + WebSocket simulated stream

Deploy: Render/Railway + Docker

Pros: Fast to build, easy demo, judges see live dashboard

Cons: Less "deep learning" novelty — compensate with strong explainability + spatial logic

48h build order:

1. Python anomaly script (standalone)
2. Node API wrapping Python via `child_process` or REST
3. JS dashboard with live charts
4. Inject 3 anomaly scenarios

Option 2: ML-Heavy Python Stack (Best for accuracy score)

Best for: Teams with ML/Python strength

Frontend: Streamlit OR React + Plotly

Backend: FastAPI + WebSocket

ML:

- LSTM Autoencoder (TensorFlow/Keras) — temporal patterns
- Isolation Forest — multivariate outliers
- Ensemble voting (both must agree = high confidence)
- SHAP + LIME

Data: Pandas pipeline, feature engineering (rolling stats, lag features)

Viz: Seaborn + Plotly dashboards

Deploy: Docker Compose

Pros: Highest detection accuracy potential; LSTM-AE is research-backed

Cons: Training time; harder to debug in 24h

Key features to engineer:

- Rolling mean/std (24h window)
- Rate of change (°C/hour)
- T–RH dew point residual
- Hour-of-day + month (seasonality)

Option 3: Edge-Cloud Hybrid (Best for Innovation + Energy Efficiency)

Best for: Teams with hardware/IoT member

Edge: ESP32 + DHT22/BMP280 sensors (or mock serial data)

- On-device rule checks (range, step, flatline counter)
- Sends flagged packets only → saves bandwidth/power

Cloud: Python FastAPI + LSTM-AE + MongoDB

Frontend: React dashboard

ML: Scikit-learn (cloud) + threshold logic (edge)

Comm: MQTT (Mosquitto broker)

Pros: Hits Energy Efficiency (5%) + Practical Deployability (10%)

Cons: Hardware debugging risk — always have a software fallback

Demo trick: Even without real ESP32, simulate edge packets via a Python script publishing to MQTT.

Tech Stack Comparison

Criterion	Option 1 (Web)	Option 2 (ML)	Option 3 (Edge)
Build speed	★★★★★	★★★	★★
Detection accuracy	★★★	★★★★★	★★★★★
Innovation score	★★★	★★★★	★★★★★
Demo reliability	★★★★★	★★★	★★★
Beginner friendly	★★★★★	★★★	★★

Recommendation: Start with Option 1 architecture, add Option 2's Isolation Forest + SHAP, and describe Option 3 on slides (edge diagram) even if not fully built.

6. Pitch Deck (Slide-by-Slide)

Use this structure for 8–10 slides, ~3 minutes:

Slide 1: Title

SkyGuard AI

Intelligent Real-Time Anomaly Detection for Automatic Weather Stations

Team name | Hackathon name | Date

Slide 2: Problem

- AWS networks power forecasts for 700M+ people
- Bad sensor data → wrong forecasts → bad decisions (agriculture, aviation, disaster)
- Real example: IMD Delhi AWS reported 52.9°C in May 2024 — faulty sensor, not reality
- Current QC = static thresholds; misses complex, multivariate faults

Slide 3: Market Insight

- IMD operates 707+ AWS stations across India
- Rule-based QC achieves ~95% accuracy — 5% error rate on critical data
- Global shift: ML autoencoders (Met Éireann, BMKG) outperform rules but not deployed at scale in India
- Gap: No explainable, real-time, multivariate QC for T + P + RH

Slide 4: Solution Overview

SkyGuard AI = 3-layer intelligent QC pipeline:

Sensor Data → [Layer 1: Rule Pre-filter] → [Layer 2: ML Anomaly Detector]

→ [Layer 3: Physics + Spatial Consistency] → Alert + Explanation

- Real-time anomaly alerts with confidence scores
- Root-cause classification (spike / flatline / drift / comm error)
- Sensor health dashboard
- Optional: corrected value estimation

Slide 5: MVP Demo

Live demo script (3 min):

1. Show normal stream (green status)
2. Inject 55°C spike → alert fires, SHAP shows "temperature spike + physics violation"
3. Inject flatline → "frozen sensor" root cause
4. Show neighbor comparison table

Slide 6: USP / Innovation

Traditional QC SkyGuard AI
Static thresholds Adaptive ML + physics rules
Single variable Multivariate T–P–RH consistency
Black box flag SHAP explainability
Reactive maintenance Sensor health score (predictive)
Cloud only Edge + cloud hybrid

Grand Challenge answer: Yes — SkyGuard is the first step toward a self-healing, self-aware AWS network.

Slide 7: Technology Stack

- ML: Python, Scikit-learn, Isolation Forest, LSTM-AE, SHAP
- Backend: FastAPI / Node.js, WebSocket, MongoDB
- Frontend: React/JS, Plotly, Chart.js
- Edge (roadmap): ESP32, MQTT
- Deploy: Docker, GitHub Actions

Slide 8: Results / Metrics

Metric Target Your Result
Precision >90% ____
Recall >85% ____
False alarm rate <5% ____
Inference latency <500ms ____
Anomaly types detected 5+ ____

Use your injected test set with known labels for honest metrics.

Slide 9: Future Roadmap

- Month 1–3: Integrate with IMD data formats (WMO BUFR)
- Month 3–6: Multi-station spatial graph neural network
- Month 6–12: ESP32 edge deployment pilots at 10 agro-AWS sites
- Year 1+: SaaS API for private AWS operators, agri-tech integrations
- Scale: 700+ stations → national QC layer

Slide 10: Team & Ask

Name Role
____ ML Engineer
____ Backend
____ Frontend / Pitch

Ask: Feedback on deployment strategy with meteorological agencies

GitHub: [link] | Live demo: [link]

7. Tips for Beginners to Win

A. Align ruthlessly with scoring weights

Weight What judges want Your move
25% Innovation Novel approach Physics-informed ML + spatial consistency + "self-
20% Accuracy Works on test data Pre-build anomaly injector; report Precision/Recal
15% Real-time Live demo WebSocket stream, not static CSV screenshots
10% Explainability SHAP/LIME Show "Why flagged?" panel on every alert
10% Scalability Architecture Multi-station diagram even if demo uses 2 stations
10% Deployability Runnable code Docker + README with python demo.py
5% UI Clean dashboard Dark theme + map + alert feed beats raw Jupyter
5% Energy Edge mention ESP32 diagram + "edge pre-filter reduces cloud cal

B. Demo > Slides > Code (priority order)

1. 3-minute live demo wins hackathons — rehearse it 5 times
2. Record a backup video in case WiFi fails
3. Slides support the demo; don't read slides
4. Code on GitHub with a one-command run (docker compose up)

C. Storytelling that resonates in India

- Open with the Delhi 52.9°C story — judges remember real incidents
- Frame as: "Protecting 700 million people's weather data"
- Connect to farmer, aviation, disaster management — not just ML metrics
- Mention IMD, Ministry of Earth Sciences, Digital India alignment

D. Technical credibility without overengineering

Do:

- Hybrid approach (rules + ML ensemble)
- Show SHAP bar chart for one alert
- Pre-compute model; demo uses inference only (fast)
- Inject known anomalies; report metrics on that set

Don't:

- Train a huge deep model live on stage
- Claim 99% accuracy without test methodology
- Build 10 features — build 4 that work perfectly
- Use buzzwords (blockchain, metaverse) with no connection

E. Judge Q&A prep (likely questions)

Question Strong answer
How is this different from IMD's existing QC? IMD uses static rules; we add adaptive ML + explai
Won't ML flag real heatwaves as anomalies? Physics consistency layer: if neighbors also show
Can this run on ESP32? Rules yes; full ML runs cloud-side; edge sends pre
What data did you use? Historical AWS-style synthetic + injected faults;
How do you measure accuracy? Labeled injected test set: X% precision, Y% recall

8. Anomaly Injection Strategy (Critical for Evaluation)

Build a script that injects these into clean data:

```
ANOMALY_TYPES = {
```

```
"SPIKE":    "Sudden +20°C in 1 reading",
```

```
"FLATLINE": "Same value for 24+ consecutive readings",
```

```
"DRIFT":    "Gradual +0.5°C/hour over 48 hours",
```

```
"COMM_GAP": "Missing data burst (NaN sequence)",
"PHYSICS_VIOLATION": "T=55°C, RH=95%, P=980hPa simultaneously",
}
```

Evaluation approach:

1. Take 1,000 clean readings
 2. Inject 50 known anomalies (10 per type)
 3. Run SkyGuard → measure TP, FP, FN, TN
 4. Present confusion matrix on Slide 8
-

9. Suggested Repository Structure

skyguard-ai/

```
├─ README.md          # Setup + demo instructions
├─ docker-compose.yml
├─ data/
│  └─ sample_aws.csv
│  └─ injected_test.csv
├─ ml/
│  └─ preprocess.py
│  └─ train.py
│  └─ detect.py        # Inference + SHAP
│  └─ anomaly_injector.py
├─ backend/
│  └─ app.py           # FastAPI
│  └─ websocket_stream.py
├─ frontend/
│  └─ index.html
│  └─ dashboard.js
│  └─ styles.css
├─ docs/
│  └─ USE_CASES.md     # Required deliverable
└─ demo/
   └─ run_demo.sh      # One-command demo
```

10. 48-Hour Priority Checklist (Print This)

- [] □ Hour 0: Scope locked, roles assigned, repo created
- [] □ Hour 4: Sample data loaded, anomaly injector working

- [] □ Hour 8: Rule-based QC + Isolation Forest detecting 3+ anomaly types
- [] □ Hour 14: SHAP explanations working
- [] □ Hour 18: API + WebSocket streaming
- [] □ Hour 22: Dashboard showing live alerts
- [] □ Hour 28: 3 demo scenarios scripted and tested
- [] □ Hour 32: Metrics computed on injected test set
- [] □ Hour 36: Pitch deck complete
- [] □ Hour 40: README + USE_CASES.md written
- [] □ Hour 44: Demo rehearsed 3x, backup video recorded
- [] □ Hour 48: SUBMIT + SLEEP

Summary: Your Winning Formula

Pillar Action
Problem Lead with Delhi 52.9°C — real, relatable, urgent
Solution Hybrid rules + ML + physics + spatial checks
Demo Live stream → inject fault → explain alert → show
Differentiation Explainability + root-cause + self-healing vision
Realism Working MVP on 3 variables, 2 stations, 5 anomaly
Presentation 3 min demo, 2 min slides, 1 min Q&A prep

Next steps (optional):

1. Generate the starter codebase (anomaly injector + Isolation Forest + dashboard skeleton)
2. Write the USE_CASES.md document for submission
3. Create slide content as a ready-to-copy markdown file for Google Slides/PPT
4. Build a synthetic AWS dataset with pre-injected anomalies for testing